

DOCUMENTATION DU MINI-PROJET

Analyseur SQL Simplifié avec Flex & Bison

Module : Langages et Traducteurs – LSTGL

Année : 2025/2026

Auteurs : CHOUAYB KASRI , HAJIR SALAH EDDINE , MERIZAK FERDAOUSSE , KHYAR IBTISSAM.

1. Introduction

Ce mini-projet consiste à réaliser un **analyseur syntaxique simplifié de SQL** à l'aide des outils **Flex** et **Bison**, utilisés pour générer automatiquement un analyseur lexical et un analyseur syntaxique.

L'objectif principal est :

- de **lire une seule requête SQL** depuis un fichier texte,
- de **reconnaître** le type de requête (SELECT, INSERT, CREATE TABLE, etc.),
- de **vérifier la validité syntaxique** de l'instruction,
- et d'afficher un **message simple décrivant la requête reconnue**.

Aucune action réelle sur une base de données n'est effectuée : le projet se concentre uniquement sur l'analyse syntaxique.

2. Cahier des charges

Le programme doit être capable de reconnaître une **unique instruction SQL** parmi les catégories suivantes :

- CREATE TABLE
- INSERT INTO
- SELECT
- UPDATE
- DELETE

- DROP TABLE

Il doit également :

- différencier un SELECT simple d'un SELECT avec clause WHERE,
- compter le nombre de valeurs dans un INSERT,
- compter le nombre de champs dans un SELECT,
- compter le nombre de modifications dans un UPDATE,
- afficher un message clair correspondant à la requête reconnue.

Le fichier d'entrée contient **une seule requête**, terminée par un point-virgule ; .

3. Architecture générale

Le projet comporte trois fichiers principaux :

- **flex.l** : l'analyseur lexical
→ Il découpe l'entrée en *tokens* (mots-clés, identifiants, constantes, opérateurs...).
- **bison.y** : l'analyseur syntaxique
→ Il définit la grammaire SQL simplifiée et les actions exécutées lors de la reconnaissance.
- **main.c** : programme principal
→ Il ouvre le fichier d'entrée, appelle l'analyseur et affiche le résultat.

Des fichiers auto-générés sont créés par Flex et Bison :

- lex.yy.c
- bison.tab.c
- bison.tab.h

L'ensemble compile en un exécutable nommé par exemple : **miniprojet**.

4. Analyseur lexical (Flex)

Le fichier **flex.l** spécifie les règles permettant de reconnaître les unités lexicales :

- **mots-clés** : SELECT, INSERT, UPDATE, CREATE...

- **types** : INT, FLOAT, VARCHAR, BOOL
- **constantes** :
 - entières (123)
 - flottantes (3.14)
 - chaînes ('texte' ou "texte")
 - booléens (TRUE, FALSE)
- **identifiants** : noms de champs ou de tables
- **opérateurs** : =, !=, <, >, <=, >=
- **symboles** : parenthèses, virgules, point-virgule, etc.
- **comments** : -- ou /* ... */
- gestion des **numéros de ligne** (yylineno)

Chaque lexème reconnu renvoie un token à Bison.

5. Analyseur syntaxique (Bison)

Le fichier **bison.y** contient :

5.1. Le symbole de départ

%start requete

Ce qui garantit que l'analyse porte **sur exactement une seule requête**.

5.2. Les tokens reconnus

Mêmes tokens que dans Flex : identifiants, types SQL, opérateurs, etc.

5.3. La grammaire

Elle définit les constructions suivantes :

- **CREATE TABLE**
- **INSERT INTO**
- **SELECT**
- **UPDATE**

- **DELETE**
- **DROP TABLE**

Chaque règle exécute un printf lorsque la requête correspondante est reconnue.

Par exemple :

```
printf("INSERT INTO %s : %d valeurs\n", $3, $6);
```

5.4. Gestion des conditions

Les expressions logiques (AND, OR) et les comparaisons (=, >=, <, !=) sont supportées.

6. Programme principal (main.c)

Le programme :

1. lit un fichier passé en argument (ou l'entrée standard),
2. lance **yyparse()**,
3. affiche le message généré par Bison,
4. annonce la fin de l'analyse.

Il ne traite qu'une seule requête par exécution, conformément au cahier des charges.

7. Compilation

Les commandes pour générer l'exécutable :

```
flex flex.l  
bison -d bison.y  
gcc lex.yy.c bison.tab.c main.c -o miniprojet
```

8. Exemples d'utilisation

Fichier d'entrée :

```
SELECT nom, age FROM Client;
```

Exécution :

```
./miniprojet test.miniprojet
```

Affichage :

```
SELECT simple, table=Client, nb_champs=2  
=== Fin de l'analyse SQL ===
```

Autres exemples :

Requête	Message affiché
INSERT INTO Client VALUES (1,'Zak',3.14);	INSERT INTO Client : 3 valeurs
DELETE FROM Client;	DELETE total table=Client
UPDATE Client SET age=22 WHERE id=1;	UPDATE table=Client : 1 modifications
DROP TABLE Client;	DROP TABLE Client

9. Limitations

- Le programme n'exécute pas les requêtes.
 - Il ne lit qu'une seule instruction par fichier.
 - SQL est volontairement limité .
 - Pas de gestion d'erreurs contextuelles (ex : table inexistante).
 - Pas de structure de données mémoire.
-

10. Conclusion

Ce mini-projet permet d'apprendre :

- la construction d'un analyseur lexical avec Flex,
- la définition d'une grammaire avec Bison,
- la communication entre les deux outils,
- la reconnaissance syntaxique d'un sous-ensemble de SQL.

Il constitue une introduction pratique aux techniques de compilation et d'analyse syntaxique.

11. Annexes

Code source

– **flex.l** :



– **bison.y** :

– **main.c** :

Exemples de fichiers .miniprojet

????????????????????????????